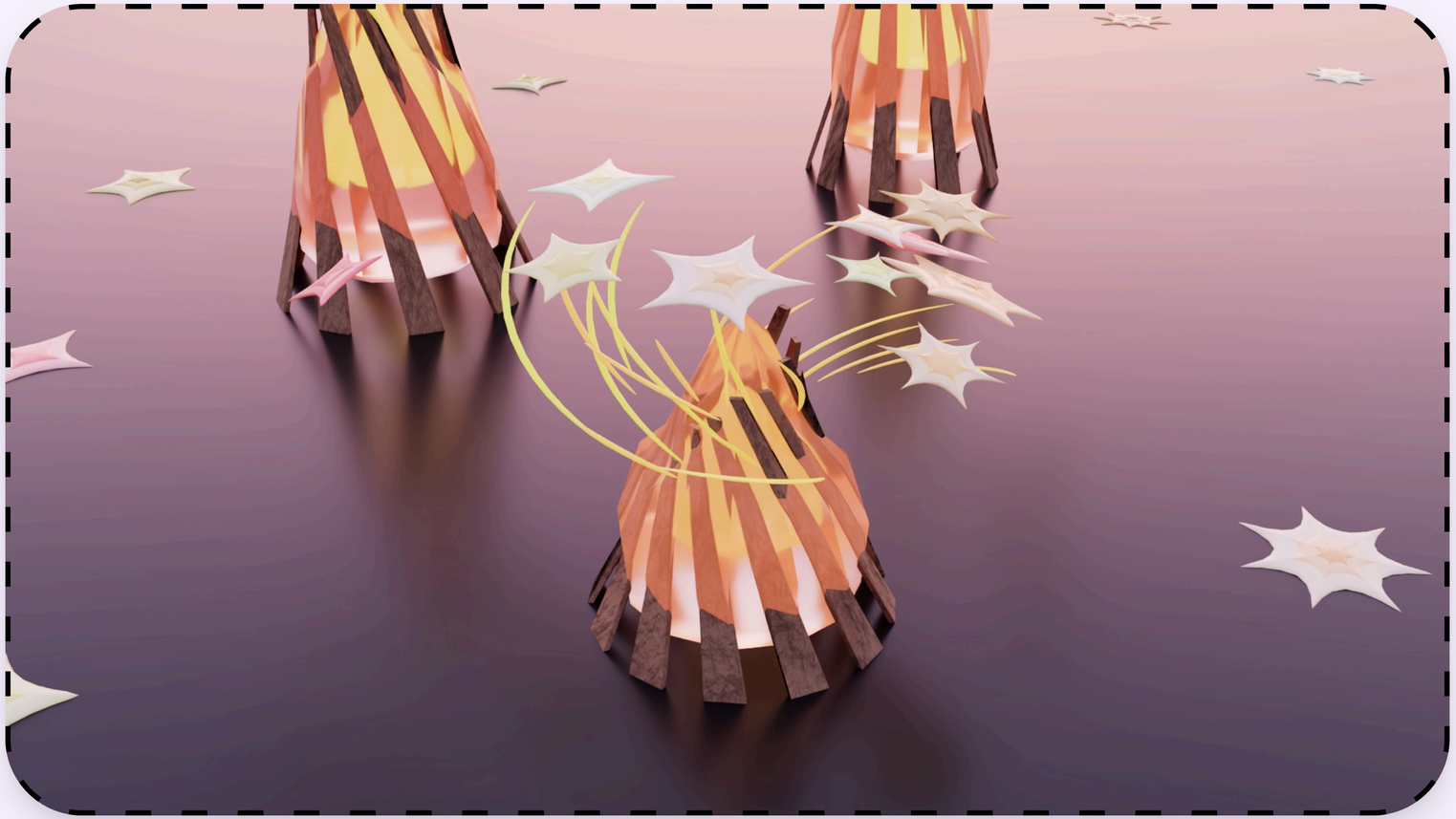
VFX 705 - 02

PROJECT 04: Tree Scattering

Lu Zhang · Mar 8th 2026

Reference



Breakdown

①

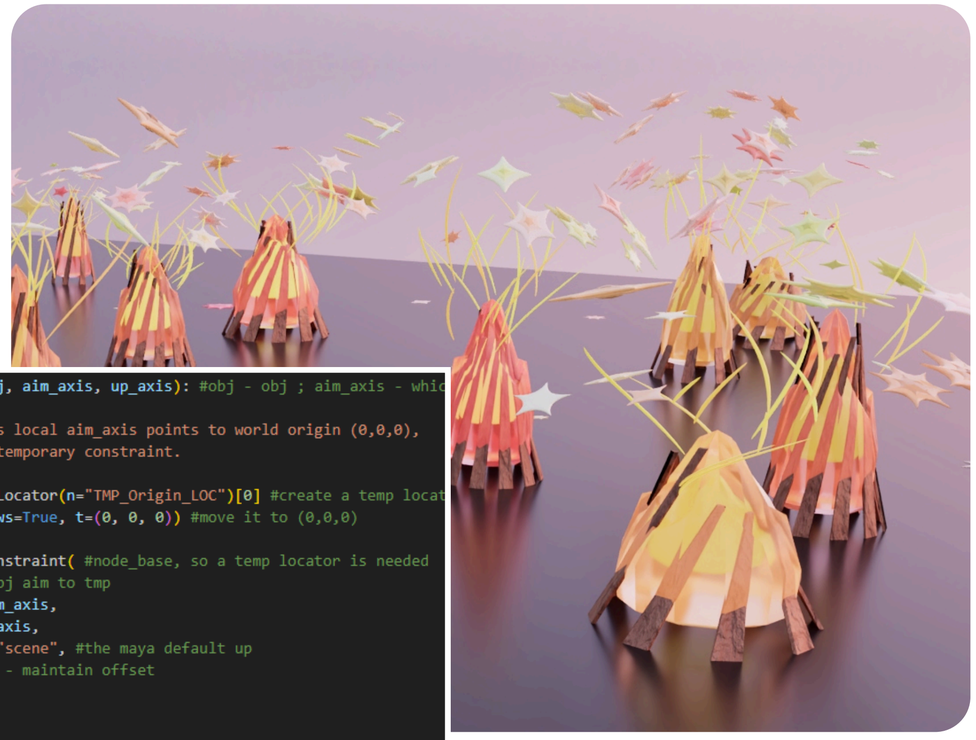
Function: aim_to_origin(obj, aim_axis, up_axis)

This function is used to automatically orient an object so that one of its local axes points toward the world origin (0,0,0). It is designed to help generated elements consistently face the center of the object.

1. A temporary locator is created and positioned at the world origin. Since Maya's constraint system requires a scene object as the target, the locator serves as a reference point that other objects can aim toward.

2. An aimConstraint is applied so that a specified local axis of the object points toward the locator. The aimVector defines which axis should face the target, while the upVector maintains a stable orientation. The world up type is set to "scene" so the orientation follows Maya's default up direction.

3. The temporary constraint and locator are deleted. Although the constraint is removed, the rotation result remains applied to the object. This keeps the scene clean while preserving the desired orientation.



```python
def aim_to_origin(obj, aim_axis, up_axis): #obj - obj ; aim_axis - whic
    """
    Rotate obj so its local aim_axis points to world origin (0,0,0),
    then delete the temporary constraint.
    """
    tmp = cmds.spaceLocator(n="TMP_Origin_LOC")[0] #create a temp locat
    cmds.xform(tmp, ws=True, t=(0, 0, 0)) #move it to (0,0,0)

    con = cmds.aimConstraint( #node_base, so a temp locator is needed
        tmp, obj, #obj aim to tmp
        aimVector=aim_axis,
        upVector=up_axis,
        worldUpType="scene", #the maya default up
        mo=False #mo - maintain offset
    )[0]

    cmds.delete(con)
    cmds.delete(tmp)
```

Breakdown

②

Random attributes - Sparks and Stems

The sparks and stems are generated with controlled color ranges to maintain visual consistency. Spark colors vary from red to yellow-green, while the stems range from orange to yellow. Their quantity, position, and scale are randomized to introduce variation and make the result feel more organic. In addition, the stems are given a twist deformation with a randomized angle to enhance the dynamic and energetic appearance of the fire. To ensure that the elements visually radiate from the campfire, the function aim_to_origin(obj, aim_axis, up_axis) is used to orient each object so that it points toward the center of the scene.

To control the number of spikes in the spark shape, I used the sx value of a polygon cylinder as the main parameter. By testing cylinders with different subdivision settings, I identified a consistent pattern in the side-edge indices and used those edges as the basis for scaling. I randomly generated a cyl_sx value between 4 and 8, then created a cylinder with sx = cyl_sx * 2. After that, I selected a set of evenly distributed side edges and scaled them inward and outward to form the pointed spark structure. After generating the spark shapes, I added smaller sparks inside the bigger ones to increase visual complexity.

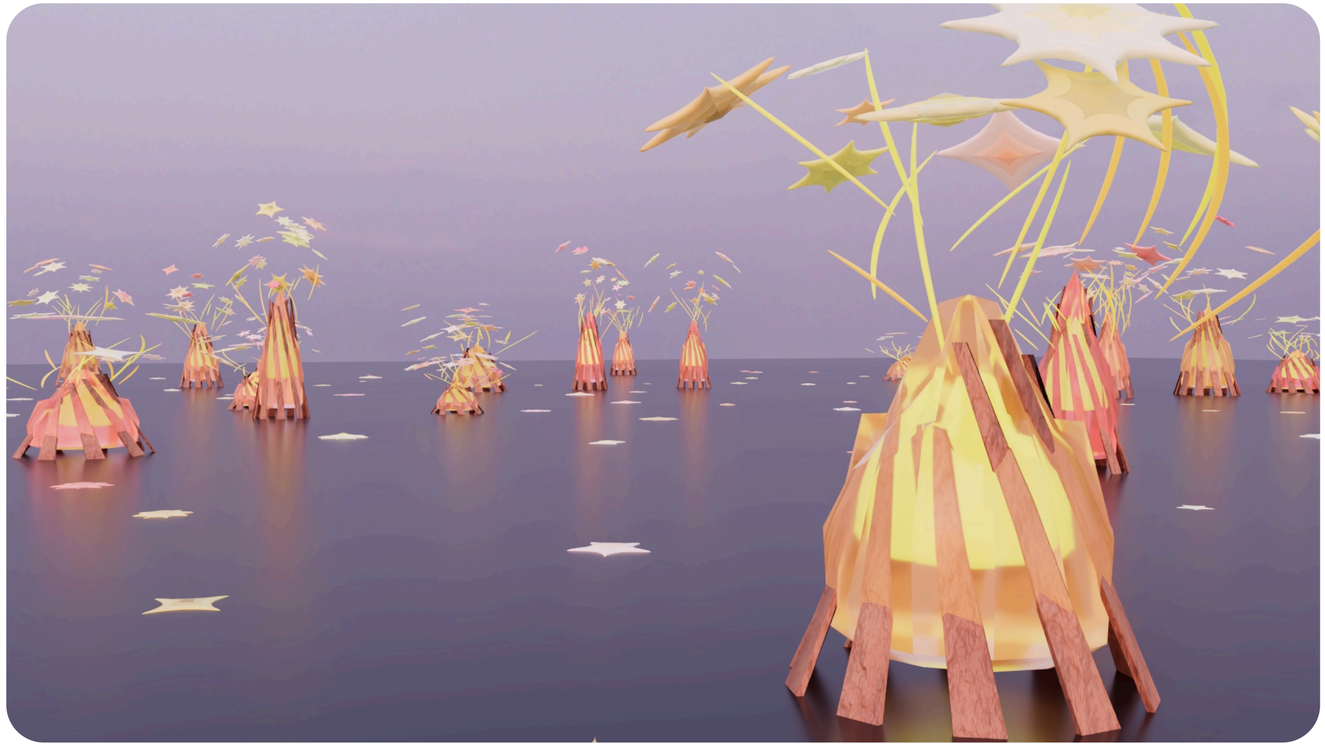
```
edges = [f"{c}.e[{cyl_sx*4+2*j}]" for j in range(cyl_sx)]
edgesc = uniform(0.01,0.2)
cmds.scale(edgesc,1.5,edgesc,*edges, r=True)
```

Breakdown

③

Random attributes - Firewood

The firewood base is created using a set of rectangular cubes arranged in a circular pattern. The number of logs is randomized between 7 and 11, and their height is also randomly varied to avoid uniformity. A twist deformer is applied to the entire group, and the twist angle is randomized.
Inside the firewood structure, two layers of flames are generated using cone geometry. Both flame layers use shaders with randomized color values, producing variations within warm fire tones. Emission and transmission parameters are also applied to give the flames a glowing appearance.



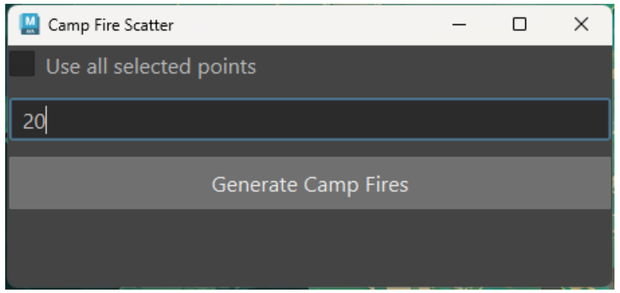
Breakdown

④

get_points(select = False, num_points): and GUI

1. A GUI called Camp Fire Scatter was created to control how the campfires are generated. The interface allows the user to choose whether to use all selected vertices or only a subset of them.

2. When select = False, the function returns two sets of sampled vertices. The first set is used for placing campfires, while the second set is sampled from the remaining vertices and used to generate sparks. This allows the system to create fewer campfires but a denser distribution of sparks.



Problems

When creating the GUI for the campfire scattering tool, I encountered an issue when connecting the button command to the main generation function.

```
# Error: main() takes 0 positional arguments but 1 was given
```

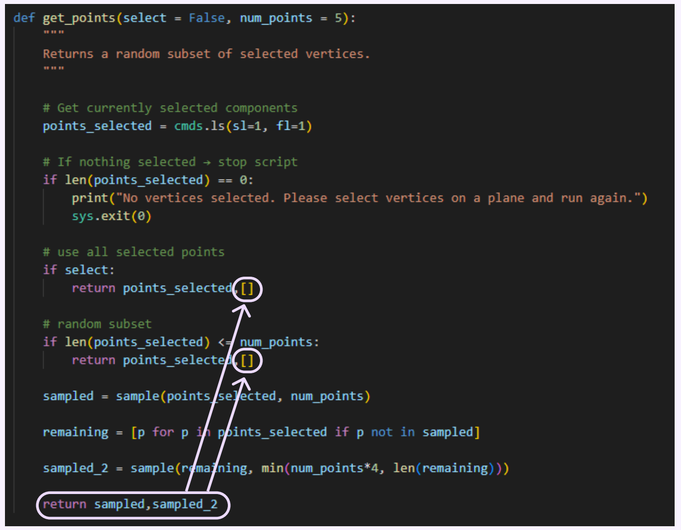
In Maya, when a button is created using cmds.button, the command automatically passes a parameter to the function when the button is clicked. However, my main() function was originally written without any parameters, which caused an error when the button attempted to execute it.

To solve this issue, I modified the function definition to:
def main(*args):

The *args parameter allows the function to accept any extra arguments passed by the GUI button. Since the script does not actually need those values, the arguments are simply ignored. This ensures that the function can be triggered safely from the interface without causing a runtime error.

While testing the GUI, an error occurred when the “Use all selected points” option was enabled.

```
# Error: ValueError: file <maya console> line 83: No object matches name: m
```



```
def get_points(select = False, num_points = 5):
    """
    Returns a random subset of selected vertices.
    """

    # Get currently selected components
    points_selected = cmds.ls(sl=1, fl=1)

    # If nothing selected → stop script
    if len(points_selected) == 0:
        print("No vertices selected. Please select vertices on a plane and run again.")
        sys.exit(0)

    # use all selected points
    if select:
        return points_selected,[]

    # random subset
    if len(points_selected) <= num_points:
        return points_selected,[]

    sampled = sample(points_selected, num_points)

    remaining = [p for p in points_selected if p not in sampled]

    sampled_2 = sample(remaining, min(num_points*4, len(remaining)))

    return sampled,sampled_2
```

The problem was caused by the get_points() function returning only one list when this option was selected, while the main() function expected two lists. I fixed this by modifying the function so it always returns a tuple of two lists (points_selected, []). After this change, the GUI worked correctly and the objects were generated without errors.

Future Improvement

If I had more time to further develop this project, I would try to make some more interesting campfires with the different themes such as seasons.